

API PERFORMANCE MONITOR

Francisco Aragão Dias
Martim Ferreira

Supervisor: Pedro Pereira

Final Project Report for the course Project and Seminar
Bachelor's Degree in Computer Science and Computer Engineering

May 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

API PERFORMANCE MONITOR

51562 Francisco Barradas Aragão de Andrade Dias

51755 Martim Afonso Almeida Ferreira

Supervisor: Pedro Alexandre de Seia e Cunha Ribeiro Pereira

Final Project Report for the course Project and Seminar
Bachelor's Degree in Computer Science and Computer Engineering

May 2026

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Professor Pedro Alexandre de Seia e Cunha Ribeiro Pereira, for his guidance, availability, valuable feedback, and continuous support throughout the development of this project. His technical expertise, constructive suggestions, and encouragement were fundamental in helping us overcome challenges and make informed decisions during the design and implementation of the system.

We are also grateful to the teaching staff of the Bachelor's Degree in Computer Science and Computer Engineering at Instituto Superior de Engenharia de Lisboa, whose courses provided the knowledge and skills that made this project possible. The concepts acquired throughout the degree served as the foundation for the work presented in this report.

A special acknowledgment is extended to our families and friends for their patience, understanding, encouragement, and unwavering support throughout this academic journey. Their motivation and confidence in our abilities were essential during the most demanding stages of the project.

Finally, we would like to thank everyone who, directly or indirectly, contributed to the successful completion of this work. Their support, advice, and encouragement played an important role in the achievement of this project.

Abstract

The objective of this project is the development of an HTTP API monitoring and observability platform named *API Performance Monitor*. The platform enables the monitoring of web service availability, performance, and behavior through the periodic collection of operational metrics such as latency, HTTP status codes, and availability information.

The proposed solution combines a distributed architecture composed of a backend developed using Kotlin and Ktor, a multiplatform client application built with Compose Multiplatform, asynchronous workers responsible for executing monitoring tasks, and a PostgreSQL database for the persistence of metrics and configuration data.

The platform supports the monitoring of public APIs and also includes support for local monitoring agents, enabling the observation of private or internal services while preserving sensitive infrastructure information.

At the current stage of development, several core functionalities have already been implemented, including user authentication, monitored endpoint management, metric collection, alert processing, PostgreSQL integration, automated testing, and API documentation through OpenAPI/Swagger.

The project follows an incremental development approach with a strong focus on modularity, maintainability, code reuse, testability, and multiplatform support.

Keywords: observability; API monitoring; metrics; Kotlin; Ktor; Compose Multiplatform; PostgreSQL; distributed systems.

Resumo

O presente projeto tem como objetivo o desenvolvimento de uma plataforma de monitorização e observabilidade de APIs HTTP, denominada *API Performance Monitor*. A plataforma permite monitorizar a disponibilidade, o desempenho e o comportamento de serviços web através da recolha periódica de métricas operacionais, incluindo latência, códigos de estado HTTP e informação de disponibilidade.

A solução proposta assenta numa arquitetura distribuída composta por um backend desenvolvido em Kotlin com Ktor, uma aplicação cliente multiplataforma baseada em Compose Multiplatform, workers assíncronos responsáveis pela execução das verificações periódicas e uma base de dados PostgreSQL para persistência de métricas e configurações.

O sistema suporta a monitorização de APIs públicas e prevê ainda a utilização de agentes locais para monitorização de serviços privados ou internos, permitindo preservar informação sensível dentro da infraestrutura do utilizador.

Nesta fase do desenvolvimento já se encontram implementadas várias funcionalidades fundamentais, incluindo autenticação de utilizadores, gestão de endpoints monitorizados, recolha de métricas, processamento de alertas, integração com PostgreSQL, testes automatizados e documentação da API através de OpenAPI/Swagger.

O projeto segue uma abordagem de desenvolvimento incremental, com especial foco na modularidade, manutenção, reutilização de código, testabilidade e suporte multiplataforma.

Palavras-chave: observabilidade; monitorização de APIs; sistemas distribuídos; recolha de métricas; Kotlin; Ktor; Compose Multiplatform; PostgreSQL.

Use of Artificial Intelligence Tools

Generative artificial intelligence tools were used solely to support the preparation of this report, including language revision, grammar correction, and text formatting.

All technical content, implementation decisions, system architecture, testing procedures, and project results were developed and validated by the authors.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Document Structure	2
2	Background and Technological Context	3
2.1	Monitoring and Observability	3
2.2	Distributed Architectures and Asynchronous Monitoring	4
2.3	Technologies Used	4
2.4	Related Systems	5
3	System Architecture and Implementation	7
3.1	System Overview	7
3.2	Backend Architecture	8
3.3	Shared Multiplatform Module	9
3.4	Monitoring Workers	9
3.5	Client Application	10
3.6	Database Design	10
3.7	API Documentation	11
4	System Validation and Current Results	15
4.1	Validation Scope	15
4.2	Backend Validation	15
4.3	Database Validation	17
4.4	Current Limitations	17
4.5	Future Work	17
	References	19

List of Figures

3.1	Overall system architecture.	8
3.2	Monitoring worker execution flow.	11
3.3	Current database model.	12
3.4	Generated OpenAPI/Swagger documentation interface.	13
3.5	Swagger/OpenAPI documentation interface.	13
4.1	Backend automated test structure.	16

List of Tables

2.1	Main technologies used in the project.	5
3.1	System components and responsibilities.	7
4.1	Automated test categories.	15

Chapter 1

Introduction

1.1 Context and Motivation

HTTP APIs play a fundamental role in modern software architectures, acting as the primary communication mechanism between applications, services, and distributed systems. The increasing adoption of microservice-based architectures has significantly expanded the dependency on APIs to ensure interoperability between system components.

In this context, continuous API monitoring has become essential to guarantee service availability, reliability, and performance. Issues such as high latency, intermittent failures, or service downtime may directly impact dependent systems and end users.

Beyond traditional monitoring, the concept of observability aims to provide deeper insight into system behavior through the analysis of operational metrics, logs, and events, enabling faster anomaly detection and incident diagnosis.

1.2 Problem Statement

Although several monitoring platforms currently exist, many solutions present limitations regarding configuration complexity, infrastructure requirements, lack of flexibility, or difficulty monitoring private and internal services.

Additionally, simple uptime verification is often insufficient to fully understand the behavior of distributed systems. Modern monitoring solutions require historical metrics, trend analysis, alerting mechanisms, and support for both public and private APIs.

This creates the need for a platform capable of combining continuous monitoring, metrics collection, alerting, and observability features within a modular and extensible architecture.

1.3 Objectives

The primary objective of this project is the development of an HTTP API monitoring and observability platform named *API Performance Monitor*.

The platform allows users to register HTTP endpoints, execute periodic checks, and collect

operational metrics such as:

- latency;
- HTTP status codes;
- availability;
- historical execution data.

The system architecture is composed of:

- a backend developed using Kotlin and Ktor;
- asynchronous monitoring workers;
- a multiplatform client application built with Compose Multiplatform;
- a PostgreSQL database for persistence;
- support for local monitoring agents.

Additionally, the platform supports configurable alert rules and notifications, allowing rapid detection of failures or performance degradation in monitored services.

The project follows an incremental development approach with strong focus on modularity, scalability, code reuse, testability, and multiplatform support.

1.4 Document Structure

This report is organized as follows:

- Chapter 2 presents the theoretical and technological background of the project;
- Chapter 3 describes the system architecture, implementation details, and developed features;
- Chapter 4 presents the testing and validation process;
- Finally, the concluding chapter summarizes the project and discusses future work.

Chapter 2

Background and Technological Context

This chapter presents the theoretical and technological context of the project, introducing concepts related to API monitoring, observability, distributed systems, and the technologies used throughout the development of the platform.

2.1 Monitoring and Observability

Modern software systems increasingly rely on distributed architectures and microservice-based communication models [1, 2]. In this context, HTTP APIs have become a critical component of modern infrastructures, acting as the primary communication layer between applications and services.

As systems grow in complexity, continuous monitoring becomes essential to guarantee service availability, reliability, and performance. Monitoring platforms are responsible for collecting operational data that allows developers and operators to understand system behavior and quickly detect failures or abnormal conditions.

Typical monitoring metrics include:

- service availability;
- request latency;
- response times;
- HTTP status codes;
- error rates.

Beyond traditional monitoring, the concept of observability aims to provide deeper insight into system internals through the analysis of metrics, logs, and events [3, 4]. Observability enables better understanding of distributed system behavior and significantly improves incident diagnosis and troubleshooting capabilities.

Within the context of HTTP APIs, observability platforms allow developers to identify performance degradation, intermittent failures, downtime events, and abnormal request patterns over time.

2.2 Distributed Architectures and Asynchronous Monitoring

Modern monitoring platforms commonly adopt distributed architectures to improve scalability, modularity, and separation of responsibilities.

In this type of architecture, independent components are responsible for specific tasks such as metrics collection, request execution, data persistence, alert processing, or visualization.

The monitoring system developed in this project relies on asynchronous monitoring workers responsible for executing periodic HTTP requests against registered endpoints [5].

This architecture provides several advantages:

- reduced coupling between system components;
- improved scalability;
- non-blocking monitoring execution;
- easier future distribution of monitoring workloads.

Additionally, the platform is being designed to support local monitoring agents capable of monitoring private APIs and internal infrastructure without exposing sensitive services to the public internet.

2.3 Technologies Used

The project was primarily developed using technologies from the Kotlin ecosystem.

The backend service was implemented using Kotlin and Ktor [6, 7]. Ktor is a lightweight asynchronous framework designed for REST API development and coroutine-based applications. This approach allowed the development of a modular and flexible backend architecture with minimal framework overhead.

The client application was developed using Compose Multiplatform [8], enabling shared UI development across multiple platforms while maintaining a modern declarative programming model.

For data persistence, the system uses PostgreSQL [9] together with Exposed ORM [10], providing type-safe database access integrated with Kotlin coroutines.

Authentication and authorization are implemented using JSON Web Tokens (JWT) [11], enabling stateless authenticated communication between client and server components.

The platform also integrates OpenAPI/Swagger for automatic API documentation and endpoint exploration [12, 13].

Table 2.1: Main technologies used in the project.

Technology	Purpose	Layer
Kotlin	Main programming language	Backend / Frontend
Ktor	REST API framework	Backend / Frontend
Compose Multiplatform	User interface	Frontend
PostgreSQL	Data persistence	Database
Exposed ORM	Database access layer	Backend
OpenAPI/Swagger	API documentation	Backend
JWT	Authentication	Backend

2.4 Related Systems

Several monitoring and observability platforms currently exist, including:

- Prometheus [14];
- Grafana [15];
- Datadog [16];
- New Relic [17];

These platforms provide monitoring, visualization, and alerting capabilities for distributed systems and services. However, many existing solutions either target enterprise environments or require significant configuration and infrastructure complexity.

The *API Performance Monitor* project aims to provide a simpler and more focused solution centered around HTTP API monitoring while simultaneously supporting:

- continuous endpoint monitoring;
- metrics collection;
- configurable alerting;
- multiplatform client applications;
- private API monitoring through local agents.

Additionally, the project prioritizes modularity, extensibility, reusable shared code, and clean system architecture.

Chapter 3

System Architecture and Implementation

3.1 System Overview

The *API Performance Monitor* platform was designed using a modular distributed architecture composed of multiple independent components responsible for monitoring, persistence, visualization, and alert processing.

The system is primarily divided into the following components:

- a multiplatform client application;
- a backend REST API;
- asynchronous monitoring workers;
- a PostgreSQL persistence layer;
- local monitoring agents for private infrastructure.

Figure 3.1 illustrates the overall architecture of the platform.

Table 3.1: System components and responsibilities.

Component	Responsibility
Client Application	User interaction and visualization of monitoring data
Backend API	Authentication, endpoint management and alert processing
Monitoring Workers	Periodic endpoint monitoring and metric collection
PostgreSQL Database	Persistent storage of monitoring data
Local Agents	Monitoring of private or internal services

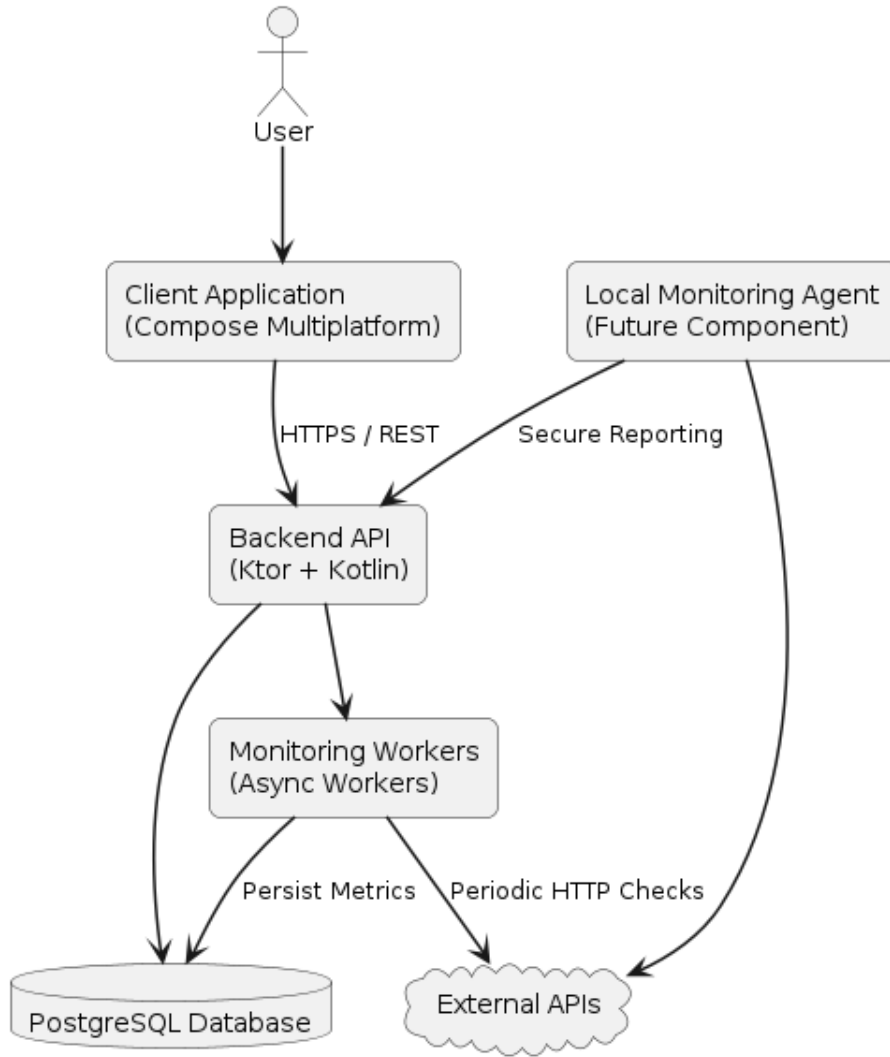


Figure 3.1: Overall system architecture.

The client application communicates with the backend API through HTTP REST endpoints using JSON payloads. The backend is responsible for authentication, endpoint management, metrics aggregation, alert processing, and persistence.

Monitoring workers periodically execute HTTP requests against registered endpoints and store collected metrics in the database.

The architecture was designed with scalability and extensibility in mind, allowing future distribution of monitoring workloads across multiple worker instances.

3.2 Backend Architecture

The backend service was implemented using Kotlin and Ktor [6, 7]. Ktor was selected due to its lightweight asynchronous architecture and native integration with Kotlin coroutines.

The application follows a layered architecture separating routing, business logic, persistence, and infrastructure concerns.

The backend is responsible for:

- user authentication;
- endpoint management;
- metrics aggregation;
- alert rule execution;
- notification dispatching;
- persistence management.

Authentication is implemented using JSON Web Tokens (JWT) [11], enabling stateless authenticated communication between the client application and backend services.

The persistence layer uses PostgreSQL together with Exposed ORM [9, 10], providing type-safe database access integrated with Kotlin coroutines.

3.3 Shared Multiplatform Module

The project makes use of Kotlin Multiplatform shared modules [18], in order to reduce duplication and improve consistency between frontend and backend components.

Shared modules currently include:

- DTO definitions;
- serialization models;
- validation logic;
- shared domain structures;
- common utility functions.

This approach improves maintainability while ensuring consistency between API contracts and client-side models.

Additionally, shared code reduces implementation overhead and simplifies future platform expansion.

3.4 Monitoring Workers

Workers are implemented using Kotlin coroutines, enabling lightweight concurrent monitoring operations with minimal resource overhead.

Monitoring operations are executed by asynchronous workers responsible for periodically checking monitored endpoints.

Each worker performs HTTP requests according to the configured monitoring interval and collects operational metrics such as:

- request latency;

- HTTP status code;
- availability state;
- execution timestamps.

The use of asynchronous workers prevents monitoring operations from blocking the main backend application and allows future horizontal scalability.

Listing 3.1: Simplified monitoring worker execution loop.

```
while (true) {  
    monitoredEndpoints.forEach { endpoint ->  
        launch {  
            monitorEndpoint(endpoint)  
        }  
    }  
  
    delay(interval)  
}
```

Figure 3.2 presents the monitoring workflow executed by workers.

3.5 Client Application

The client application was developed using Compose Multiplatform.

The application provides a graphical interface allowing users to:

- authenticate into the platform;
- manage monitored endpoints;
- visualize collected metrics;
- configure alert rules;
- inspect endpoint history and availability.

The interface follows a dashboard-oriented design focused on simplicity and usability.

3.6 Database Design

The database schema is centered around three main entities:

- users;
- monitored endpoints;
- request metrics.

Relationships between entities allow associating metrics with endpoints and endpoints with individual users.

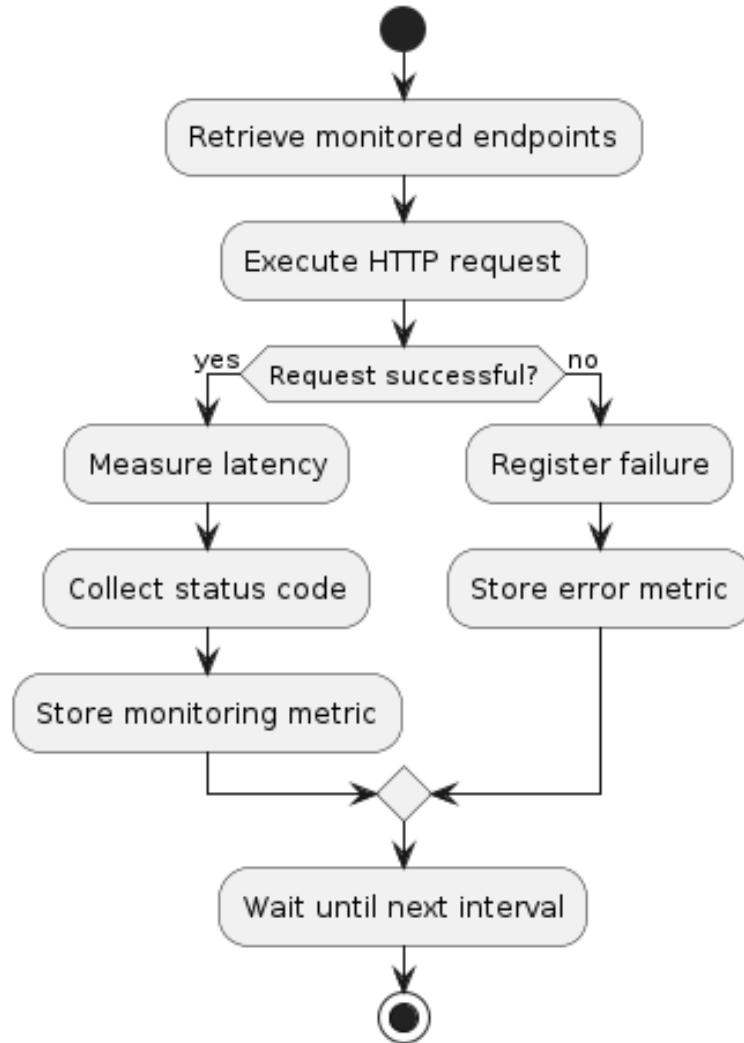


Figure 3.2: Monitoring worker execution flow.

The current persistence layer supports historical metric storage, enabling future trend analysis and dashboard visualizations.

Figure 3.3 presents the current database model.

The persistence layer was implemented using PostgreSQL together with Exposed ORM, enabling type-safe database access integrated with Kotlin coroutines.

3.7 API Documentation

The backend integrates OpenAPI/Swagger for automatic API documentation generation [12, 13]. This functionality simplifies endpoint exploration, testing, and future integration with external systems. The generated documentation also exposes authentication flows, request schemas, and monitoring endpoints, simplifying development and testing workflows.

Figure 3.4 & Figure 3.5 present the generated Swagger interface.

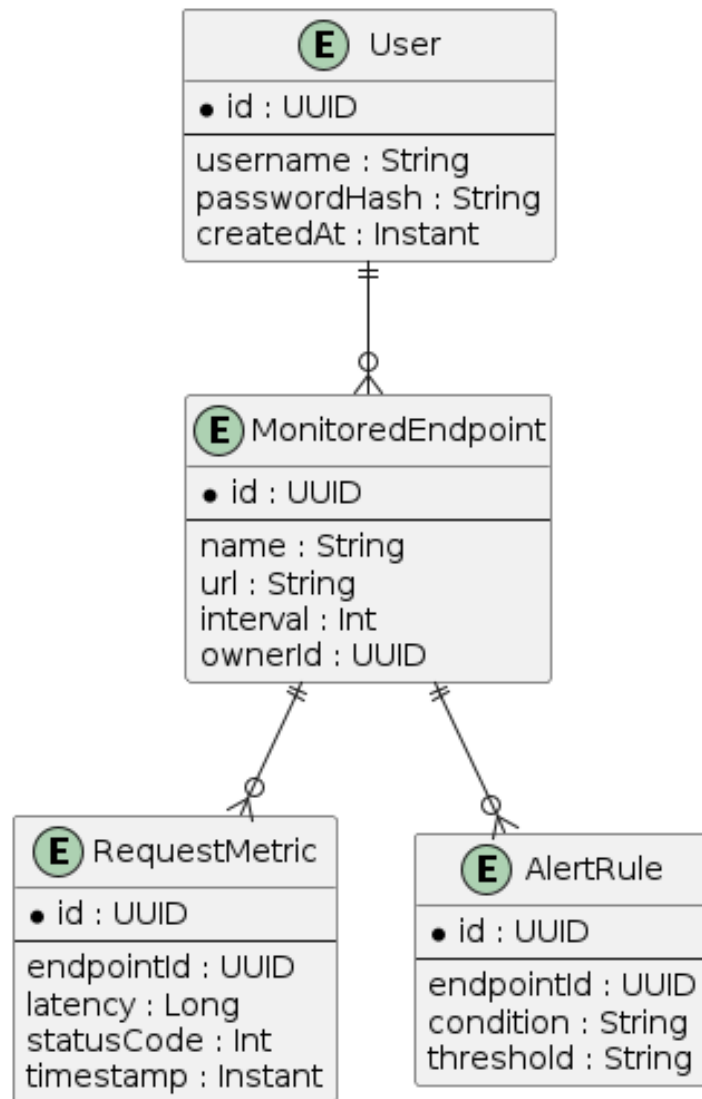


Figure 3.3: Current database model.

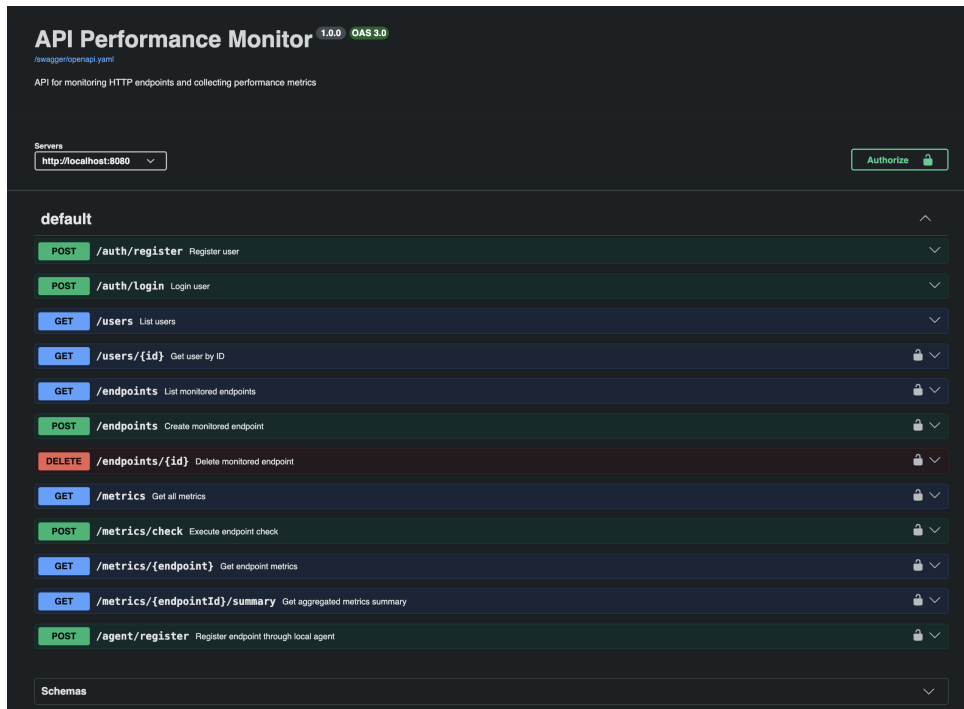


Figure 3.4: Generated OpenAPI/Swagger documentation interface.

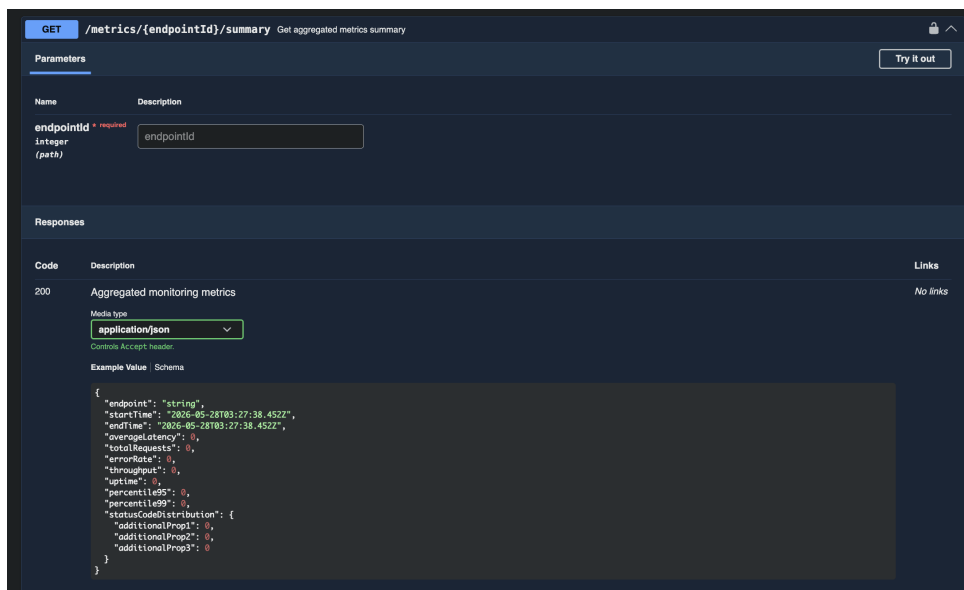


Figure 3.5: Swagger/OpenAPI documentation interface.

Chapter 4

System Validation and Current Results

4.1 Validation Scope

The validation process focused on the functionality currently available in the platform, including authentication, endpoint management, monitoring execution, metric collection, persistence, and API interaction.

The objective was to verify the correctness of the implemented functionality and ensure consistency across the different architectural layers of the system.

4.2 Backend Validation

To ensure the reliability of the platform, automated tests were developed for multiple backend layers.

The backend testing strategy covers all major architectural layers of the platform.

Repository tests validate both the PostgreSQL-based persistence implementation and the in-memory repositories used during testing. Route tests verify the behavior of the exposed REST API endpoints, while service tests validate the business logic responsible for authentication, endpoint management, monitoring operations, notifications, and metric processing.

In addition, application-level integration tests verify the interaction between the different system components in a complete execution environment.

Table 4.1: Automated test categories.

Test Category	Purpose
Repository Tests	Validate data persistence operations
Route Tests	Validate REST API endpoints
Service Tests	Validate business logic
Integration Tests	Validate interaction between components

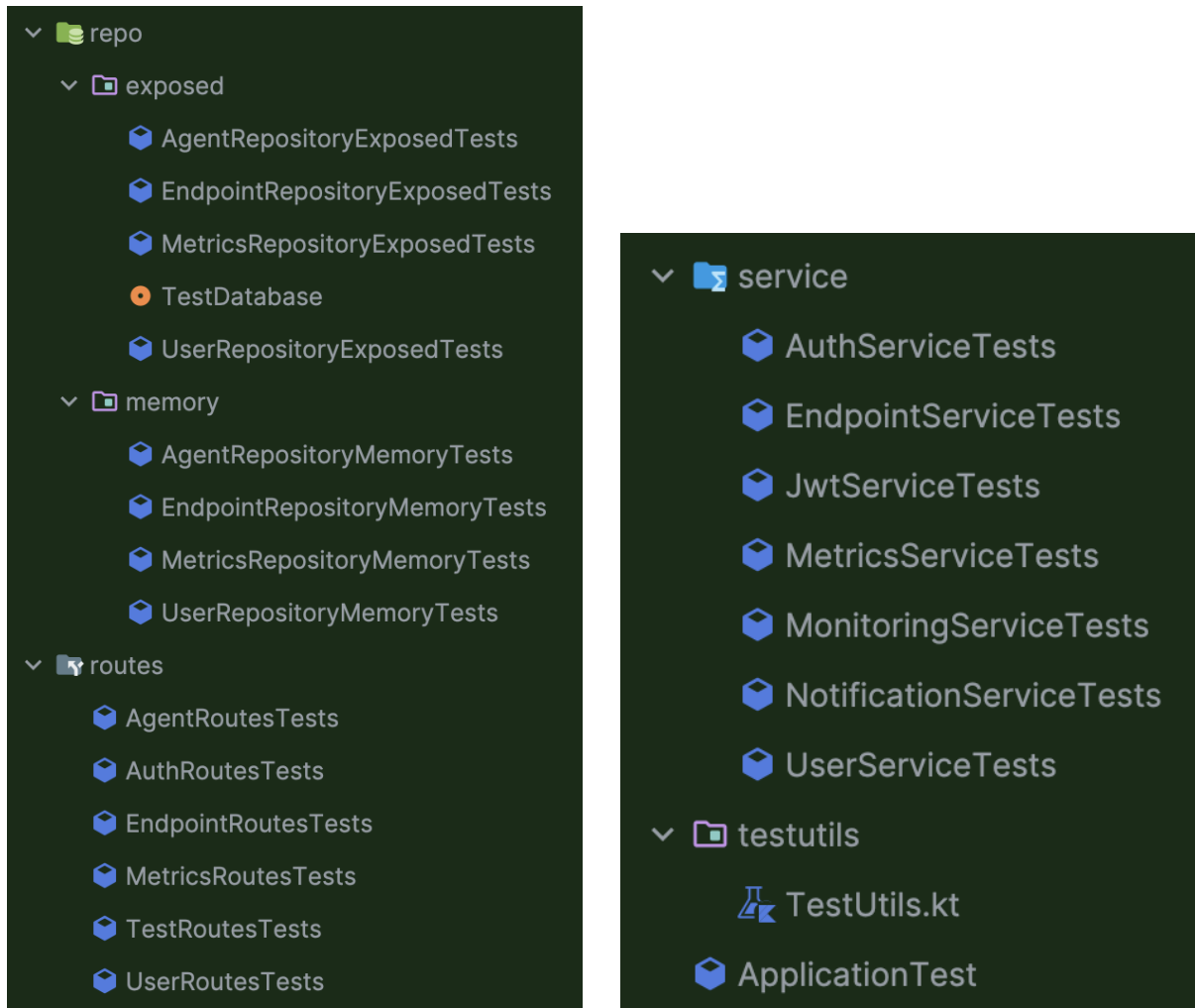


Figure 4.1: Backend automated test structure.

The current automated test suite includes:

- repository tests for users, endpoints, agents and metrics;
- REST route tests for authentication, endpoints, metrics, agents and users;
- service layer tests for authentication, JWT generation, monitoring, notifications and endpoint management;
- application-level integration testing utilities.

This testing strategy helps ensure correctness across the persistence, business logic and API layers of the system.

The automated test suite verifies several core platform features, including user authentication, endpoint management, monitoring execution, and data persistence.

The successful execution of these tests provides confidence in the correctness of the implemented functionality and supports future development and refactoring efforts.

4.3 Database Validation

Database validation was performed through repository tests and monitoring execution scenarios.

The validation process confirmed the correct creation, storage, retrieval, and association of:

- users;
- monitored endpoints;
- collected metrics;
- monitoring data;

Both in-memory and PostgreSQL repository implementations were validated through automated tests, ensuring consistent behavior across different persistence strategies.

The results confirmed that historical monitoring information is correctly persisted and can be retrieved for future analysis, reporting, and dashboard visualization.

4.4 Current Limitations

Although the platform is already functional, several features remain under development.

Current limitations include:

- dashboard visualizations are still being refined;
- local monitoring agents are not yet fully implemented;
- notification channels are currently limited;
- advanced analytics and reporting capabilities are not yet available.

4.5 Future Work

The next phase of development will focus on completing the remaining observability features and preparing the final project delivery.

Planned improvements include:

- implementation of local monitoring agents;
- advanced dashboard visualizations;
- additional notification channels;
- trend analysis and reporting tools;
- performance optimization and scalability improvements.

Bibliography

- [1] Sam Newman. *Building Microservices*. O'Reilly Media, 2nd edition, 2021.
- [2] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [3] Charity Majors, Liz Fong-Jones, and George Miranda. *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, 2022.
- [4] Dynatrace. What is observability?, 2025.
- [5] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [6] JetBrains. Kotlin documentation, 2026.
- [7] JetBrains. Ktor documentation, 2026.
- [8] JetBrains. Compose multiplatform documentation, 2026.
- [9] PostgreSQL Global Development Group. Postgresql documentation, 2026.
- [10] JetBrains. Exposed documentation, 2026.
- [11] M. Jones, J. Bradley, and N. Sakimura. RFC 7519: JSON Web Token (JWT), 2015.
- [12] OpenAPI Initiative. Openapi specification, 2025.
- [13] Swagger. Swagger documentation, 2025.
- [14] Prometheus Authors. Prometheus documentation, 2025.
- [15] Grafana Labs. Grafana documentation, 2025.
- [16] Datadog. Datadog documentation, 2025.
- [17] New Relic. New relic documentation, 2025.
- [18] JetBrains. Kotlin multiplatform documentation, 2026.